



Scénario de démonstration

QuartierConnect — parcours à suivre pour la soutenance

▶ AVANT DE COMMENCER

👤 **alice@demo.fr · résident**

🛡️ **bob@demo.fr · modérateur**

🔑 **admin@demo.fr · admin**



Ouvrir l'application

Mot de passe commun **Demo1234!** · **2FA TOTP** (secret sur la fiche imprimée) — avoir un authenticator prêt. Application : quartierconnect.duckdns.org, administration sur [/admin](#). Astuce : garder une 2^e fenêtre / navigation privée ouverte pour la messagerie temps réel.

1 Entrée résident — Alice

01 Connexion + 2FA TOTP alice · résident

[/login](#)

À FAIRE

1. Saisir **alice@demo.fr / Demo1234!** → Continuer.
2. Saisir le code TOTP à 6 chiffres (authenticator).
3. Le champ se valide seul à la 6^e touche → tableau de bord.

À MONTRER / DIRE

Aucune connexion sans **double authentification**. Le code à usage unique est vérifié côté serveur avec une **garde anti-rejeu** : un code déjà consommé est refusé.

> [POST /auth/login + TOTP replay-guard](#)

02 Onboarding — rattachement au quartier alice · résident

[/onboarding/address](#)

À FAIRE

1. Taper une adresse → autocomplétion géocodée en direct.
2. Choisir une suggestion → Confirmer.
3. Adresse reconnue → redirection automatique vers le tableau de bord.

À MONTRER / DIRE

L'adresse est géocodée puis testée par **point-dans-polygone** : dans un quartier existant → rattachement instantané ; sinon → validation modérateur.

> [/geocoding/search + point-in-polygon](#)

03

**Tableau de bord + carte communautaire**

alice · résident

/dashboard

À FAIRE

1. Montrer l'accueil personnalisé + le solde de points.
2. Pointer la carte : polygone du quartier, marqueurs colorés, domicile.
3. Cliquer un marqueur ; faire défiler les cartes (points, votes, events...).

À MONTRER / DIRE

Une seule carte **agrège toute la vie du quartier** — chaque type d'activité a sa couleur, et le marqueur domicile vient de l'adresse de l'onboarding.

[> Leaflet + polygone GeoJSON](#)

2

Signaler & s'entraider

04

**Signaler un incident + vote pour/contre**

alice · résident

/incidents

À FAIRE

1. « Signaler » → titre, description, catégorie.
2. Cliquer sur la carte pour poser l'épingle (alerte si hors quartier) → Valider.
3. Ouvrir l'incident et voter ↑ (pour) / ↓ (contre).

À MONTRER / DIRE

Le signalement se **géolocalise d'un clic**, et la communauté priorise en votant — un vote par personne, le score net s'actualise en direct.

[> POST /incidents + votes \(1/personne\)](#)

05

**Services — parcourir l'annuaire & la carte**

alice · résident

/services

À FAIRE

1. Ouvrir « Services » ; montrer la carte (épingles Offre/Demande, domicile).
2. Jouer le filtre : Toutes → Offre → Demande.
3. Faire défiler les cartes de services.

À MONTRER / DIRE

L'annuaire est **géolocalisé et cadré sur votre quartier** ; les épingles se regroupent quand on dézoome.

[> Leaflet + clustering · infinite scroll](#)

06

**Services — proposer un service payant**

alice · résident

/services

À FAIRE

1. « Proposer un service » → titre, Offre, catégorie Bricolage.
2. Type = Payant, Durée = 60 min.
3. Observer « Coût : 2 points » calculé en direct → Créer.

À MONTRER / DIRE

Le **prix en points s'affiche en temps réel** pendant la saisie (1 point / 30 min) — pas de calcul manuel.

[> computeServicePoints \(règle partagée\)](#)

07

**Services — réserver en points**

alice · résident

/services

À FAIRE

1. Repérer une annonce « Payant » d'un voisin.
2. « Réserver » → confirmer « X points débités à la signature ».
3. Redirection vers « Réservations » (onglet Envoyées).

À MONTRER / DIRE

L'économie de quartier tourne en points ; les points ne sont **débités qu'à la signature du contrat** — traçable, sans argent.

[> useCreateBooking](#) → [flux de contrat](#)

08

**Points — solde & transfert par email**

alice · résident

/points

À FAIRE

1. Ouvrir « Points » (compteur de solde).
2. Taper « bob » dans le destinataire (recherche par email).
3. Montant 5 + note « Merci ! » → Transférer.

À MONTRER / DIRE

Transfert de **pair à pair** en un geste : on cherche un voisin par email, on envoie des points avec un mot, l'historique se met à jour aussitôt.

[> transfert atomique + recherche débouçée](#)

3

Se retrouver

09

**Événements — parcourir & rejoindre**

alice · résident

/events

À FAIRE

1. Basculer Calendrier / Liste / Carte.
2. Ouvrir un événement → « Je participe ».
3. Le bouton passe « Inscrit », compteur +1.

À MONTRER / DIRE

Quatre façons de vivre l'agenda (calendrier, liste, carte, swipe) — inscription en un clic, compteur à jour.

[> useEvents + useEventInterest](#)

10

**Événements — mode Découverte (swipe)**

alice · résident

/events

À FAIRE

1. Onglet « Découverte (swipe) ».
2. Glisser à droite = intéressé (cœur), à gauche = passer.
3. Enchaîner les événements à venir.

À MONTRER / DIRE

Découverte **façon Tinder** pour l'agenda — geste tactile et souris, animation de sortie.

[> react-swipeable](#)

11

**Messagerie — démarrer une conversation**

alice · résident

/messages

À FAIRE

1. « Nouvelle conversation » → bob@demo.fr → Démarrer.
2. Écrire un message et l'envoyer.
3. Montrer pièce jointe  et message vocal .

À MONTRER / DIRE

On démarre juste avec **l'email d'un voisin** ; texte, pièces jointes et **messages vocaux** enregistrés dans le navigateur.

>_ **MediaRecorder + fichiers en URL authentifiée**

12

**Messagerie temps réel — 2 fenêtres**

alice + bob · 2 fenêtres

/messages

À FAIRE

1. 2^e fenêtre (navigation privée) connectée en **bob**, même conversation.
2. Montrer la pastille de présence ; Bob tape → « écrit... » chez Alice.
3. Envoi depuis Bob → apparaît instantanément chez Alice, sans recharger.

À MONTRER / DIRE

Le **vrai temps réel** : présence, indicateur « écrit... » et livraison instantanée entre les deux fenêtres — la démo la plus parlante.

>_ **WebSocket Socket.IO · presence / typing / new_message**

4

Fonctions avancées & sécurité

13

**Sondage — voter en un clic**

alice · résident

/votes

À FAIRE

1. Voisinage > Sondages, onglet « Ouverts ».
2. Choisir une option → Voter.
3. Bascule instantanée vers les résultats + badge « Vous avez voté ».

À MONTRER / DIRE

Vote enregistré aussitôt, barre de résultats **sans recharger** ; décomptes **anonymes** ; 4 types (binaire, choix unique/multiple, pondéré).

>_ **mutation optimiste + agrégats anonymes**

14

**Signature électronique validée par TOTP**

alice · résident

/contracts

À FAIRE

1. Mon espace > Contrats → « Signer avec TOTP ».
2. Lire les termes → dessiner la signature sur le pad.
3. Saisir le code à 6 chiffres → Signer.

À MONTRER / DIRE

Signature en **3 étapes** : lecture, tracé manuscrit, validation forte TOTP. Sans le bon code, refus serveur ; **frise d'audit** + PDF signé.

>_ **signature pad + TOTP (même garde que le login)**

15

**Recommandations personnalisées (Neo4j)**

alice · résident

[/recommandations](#)**À FAIRE**

1. Voisinage › Recommandations.
2. Parcourir services / événements / voisins suggérés.
3. Pointer la raison affichée sous chaque carte.

À MONTRER / DIRE

Suggestions issues d'un **graphe Neo4j** (Cypher croisant **LIVES_IN**, **ATTENDING**, **HELPED**) ; **fail-open** si le graphe tombe.

[> Neo4j + Cypher scoré](#)

16

**Bascule multilingue FR / EN à chaud**

alice · résident

profil

À FAIRE

1. Cliquer le profil (bas de la barre latérale).
2. Sous-menu « Langue » (globe).
3. Français → English : l'UI se traduit à chaud.

À MONTRER / DIRE

Toute l'interface bascule **sans rechargement**, formatage des dates selon la locale, choix mémorisé pour la prochaine visite.

[> i18next / react-i18next](#)

5

Administration & modération

17

**Connexion au panneau d'administration**

admin

/admin

À FAIRE

1. Aller sur [/admin](#), se connecter [admin@demo.fr](#) + TOTP.
2. Arriver sur le tableau de bord admin.
3. Montrer la barre : Communauté, Gestion, Outils.

À MONTRER / DIRE

Back-office **séparé, réservé au rôle admin** (un modérateur est redirigé). Bob modère, lui, via l'API et l'app desktop qui suit.

[> app admin distincte + garde de rôle](#)

18

**Créer un sondage communautaire**

admin

/community-votes

À FAIRE

1. Communauté › Sondages › Créer.
2. Titre, type (binaire / unique / multiple / pondéré), options, clôture → valider.
3. Montrer « Clôturer » et « Voir les résultats ».

À MONTRER / DIRE

L'envers du sondage voté par Alice : **création admin-only**, choix du scrutin et de l'échéance ; agrégation anonyme.

[> 4 types de scrutin, clôture manuelle](#)

19

**Modération des signalements**

admin

/incidents

À FAIRE

1. Communauté › Signalements ; filtrer, onglets Liste / Carte.
2. Avancer le statut : ouvert → en cours → résolu.
3. Supprimer un signalement abusif.

À MONTRER / DIRE

Machine à états stricte (aucune transition sauvage), réservée modérateur/admin ; la carte affiche tous les signalements filtrés.

[> machine à états serveur + Leaflet](#)

20

**Tracé cartographique d'un quartier**

admin

/neighborhoods

À FAIRE

1. Gestion › Quartiers › Créer.
2. Tracer un polygone au clic sur la carte.
3. Ajuster les sommets (quartiers voisins superposés) → enregistrer.

À MONTRER / DIRE

On **dessine la frontière du quartier à la main** ; polygone stocké en GeoJSON, base du rattachement d'adresse ; anti-chevauchement affiché.

[> éditeur de polygone Leaflet · GeoJSON](#)

21

**Adresses non couvertes**

admin

/uncovered-addresses

À FAIRE

1. Gestion › Couverture.
2. Liste triable des résidents hors quartier ; onglet Carte.
3. « Tracer un quartier » → renvoie vers l'éditeur.

À MONTRER / DIRE

Détection automatique des résidents hors de tout polygone (les trous de couverture) ; un clic renvoie l'admin vers le tracé.

[> contrôle point-dans-polygone](#)

22

**DSL d'administration (langage de requête)**

admin

/dsl

À FAIRE

1. Outils › DSL.
2. Taper `FIND incidents WHERE status = "open" LIMIT 10` → Exécuter (Ctrl+Entrée).
3. Montrer le tableau + le temps (ms) ; un `COUNT` et une requête invalide.

À MONTRER / DIRE

Notre **mini-langage maison** (FIND / WHERE / AND-OR / LIMIT / COUNT) parsé côté serveur → requête base ; ouvert aux modérateurs mais **scopé à leur quartier**.

[> tokenizer + parseur → MongoDB, scopé au rôle](#)

6 Desktop, documentation & clôture

23



App desktop JavaFX — hors-ligne + sync three-way

desktop · bob

application desktop

À FAIRE

1. Lancer l'app, se connecter en **bob** (modérateur).
2. Vue Signalements → interrupteur « Hors-ligne » ; créer/modifier un incident (SQLite local).
3. Repasser en ligne → « Synchroniser » ; résoudre un conflit (Garder local / Accepter serveur).

À MONTRER / DIRE

100 % hors-ligne sur SQLite local. Au retour, **merge three-way façon Git** (base/local/distant par champ) : un côté modifié → auto ; deux côtés → conflit tranché à la main.

>_ JavaFX + SQLite · ThreeWayMerger · /incidents/sync

24



Documentation — /aide, /dev, référence API

public

/aide · /dev · /scalar

À FAIRE

1. **/aide** : documentation utilisateur.
2. **/dev** : doc technique (protégée par mot de passe).
3. **/scalar** : référence API interactive — explorer un endpoint.

À MONTRER / DIRE

Trois surfaces : aide grand public, doc dev, et référence API **Scalar générée depuis le code NestJS**. **/dev** et **/scalar** sont derrière une auth basique.

>_ 2 sites Fumadocs + Scalar (OpenAPI)